Efficient inference in Bayesian Networks via: Junction Trees & Message Passing

Contents

12.1 Efficient Inference in Bayesian Networks **1**
12.1.0.1 A road to efficient inference . . . 2
12.1.1 Constructing Bayesian Networks . . . 2
12.1.2 The Junction tree algorithm . . . 3
12.1.3 Decomposable Undirected Graphs . . . 3
12.1.3.1 Cliques and separators . . . 4
12.1.3.2 Decomposable graphs . . . 4
12.1.3.3 Triangulation . . . 7
12.1.3.4 Triangulation: Examples . . . 7
12.1.4 Construction of the Junction Tree . . . 8
12.1.4.1 The running intersection property . . . 8
12.1.5 Identify cliques and separators to form a bipartite graph . . . 9
12.1.6 From bipartite graph to junction tree . . . 9
12.1.7 Inference in a junction tree . . . 10

12.2 Message Passing **11**
12.2.1 Overview . . . 11
12.2.2 Procedure . . . 11
12.2.3 The sum-product algorithm . . . 11

12.3 Message Passing Example **12**

12.4 References **14**

12.1 Efficient Inference in Bayesian Networks

- Last time:
  - What is a graphical model
  - Performing inference using the full joint distribution
  - Product rule, Bayes' Theorem
  - exploiting conditional independence
  - Bayesian Networks (topology, cond. prob. tables)
- Today:
  - the Junction tree algorithm
  - the message passing algorithm (factor analysis anyone?)

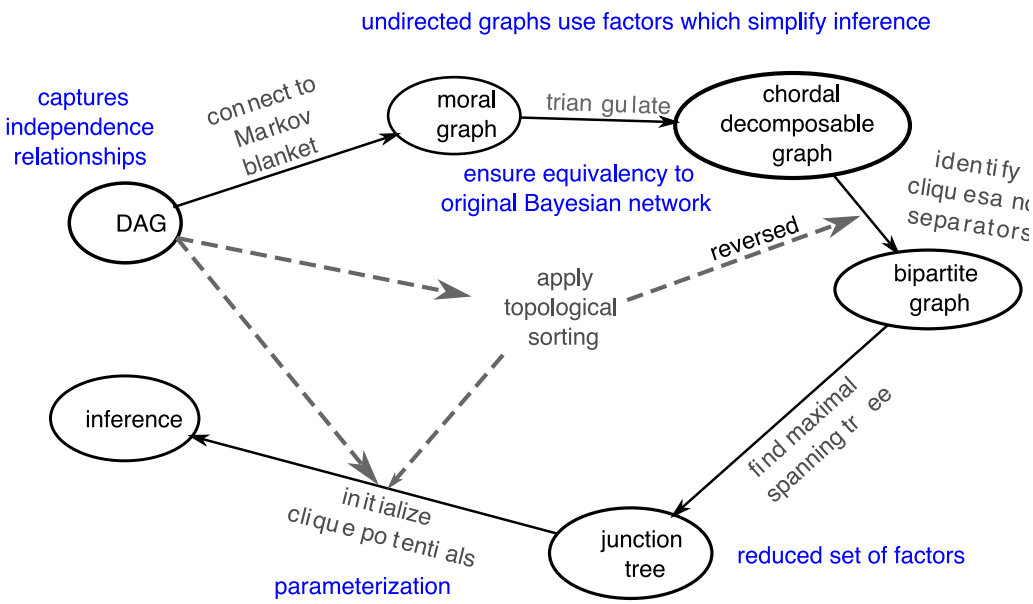


Figure 12.1.1: A road to efficient inference and a handwavy justification for the different steps

12.1.0.1 A road to efficient inference

12.1.1 Constructing Bayesian Networks

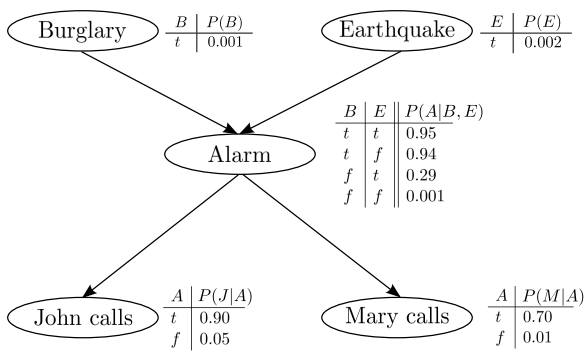


Figure 12.1.2: DAG of the Californian example

Considering statistical dependencies, only 10 instead of 31 entries have to be stored.

$$P(J, M, A, B, E) \overset{\text{product rule}}{=} P(J|M, A, B, E)\, P(M|A, B, E)\, P(A|B, E)\, P(B|E)\, P(E) \tag{12.1.1}$$

$$\overset{\text{cond. indep.}}{=} P(J|A)\, P(M|A)\, P(A|B, E)\, P(B)\, P(E) \tag{12.1.2}$$

Bayesian Networks are a factorization of the full joint distribution[1]:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i | parents(X_i)) \tag{12.1.3}$$

[1] See Ch. 14 from (Russell and Norvig, 2016) for more details.

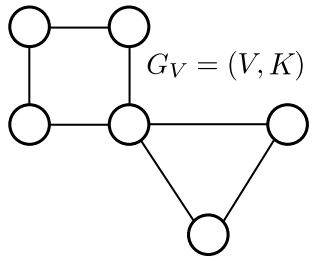


Figure 12.1.3: undirected graph G_V with vertices V and edges K

- We only need to condition on the direct parents of any node X_i
- The factorization enables us to construct the *directed acyclic graph* (DAG).
- We can also use the DAG to formulate the factorization.

$$DAG_{\text{Bayes.Net}} \quad \widehat{=} \quad \prod_{i=1}^{n} P(X_i | parents(X_i)) \tag{12.1.4}$$

12.1.2 The Junction tree algorithm

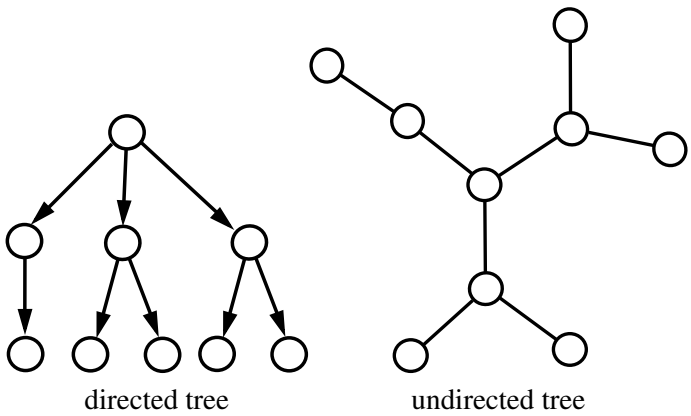
We need a representation that scales well for large DAGs to perform marginalization ("summing out") and conditioning more efficiently.

The Junction tree algorithm converts into a tree structure made up of *cliques* and *separators*.

Q 12.1.1: *What is a difference between a DAG and a tree?*

- A tree structure guarantees that the path between any two nodes is unique.

- **tree**: graph where each existing path between nodes is *unique*

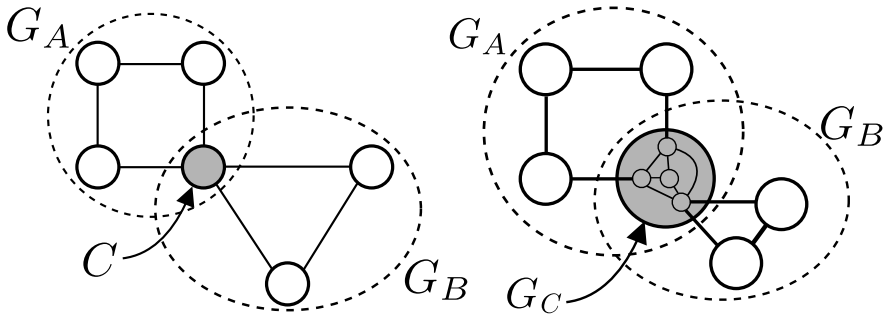


12.1.3 Decomposable Undirected Graphs

Separate G (undirected graph) into two separate subgraphs G_A and G_B , such that node C lies on the path between <u>any</u> node in G_A and <u>any</u> other node in G_B.

Remark: C can also be a set of nodes (cf.).

12.1.3.1 Cliques and separators



(a) node C (b) set of nodes C

Figure 12.1.4: subgraphs A and B separated by C

Separator

A set C of nodes **separates** two undirected subgraphs G_A and G_B if *every* path from node set A to node set B has to pass through an element of C.

12.1.3.2 Decomposable graphs

- A, B, C are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - A, B, C are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - C separates G_A and G_B, and
  - subgraph G_C is complete.
- G is **decomposable** if it is complete, or a proper decomposition A, B, C exists, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.

Q 12.1.2: *Are the following (i) a proper decomposition, (ii) decomposable graphs? (cf. Fig. 12.1.5, Fig. 12.1.6, Fig. 12.1.7 and Fig. 12.1.8)*

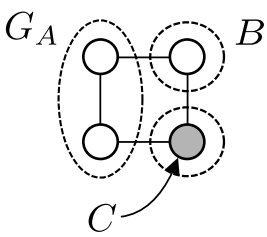


Figure 12.1.5: not a proper decomposition and not decomposable

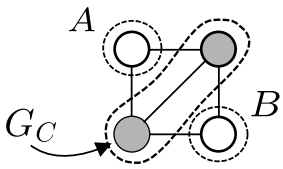
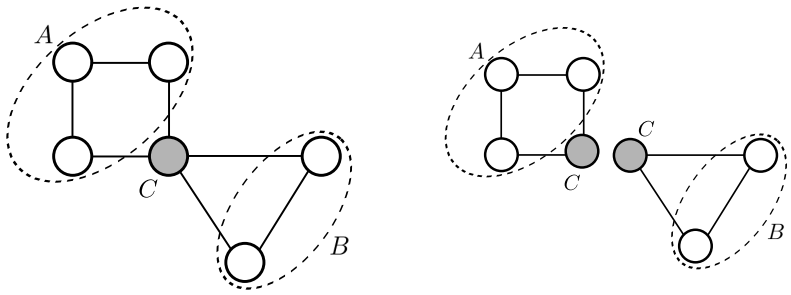


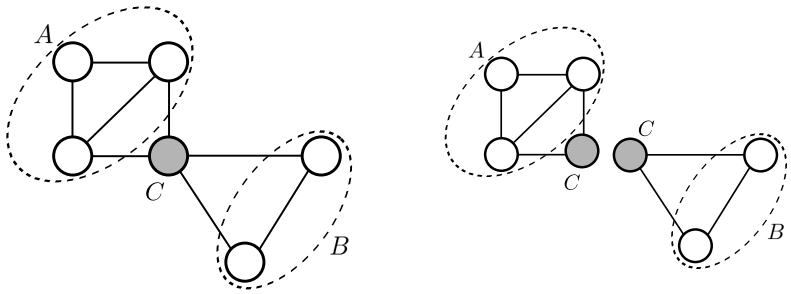
Figure 12.1.6: a proper decomposition and decomposable



(a) example decomposition

(b) Hint : because $G_{A\cup C}$ is not decomposable

Figure 12.1.7: a proper decomposition but not decomposable



(a) example decomposition

(b) Hint : because $G_{A\cup C}$ is decomposable and $G_{B\cup C}$ is complete

Figure 12.1.8: a proper decomposition and decomposable

Q 12.1.3: *Why are we interested in decomposable graphs?*

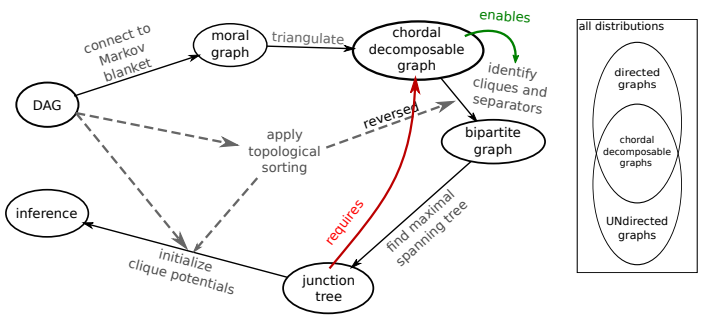


Figure 12.1.9: The construction of a junction tree requires a decomposable graph of cliques.

- The short answer[2]:

(a) it is a requirement for the construction of junction trees (cf. Section 12.1.4)

(b) decomposable graphs allow us to form a *bipartite graph* of cliques and separators.

[2]Not required but if interested in more formal justifications see Ch 3.4 and Ch 4 from (Koller and Friedman, 2009)

Cliques and separators

- **cliques** are maximally complete subgraphs
- decomposable graphs can be decomposed into cliques and separators
- cliques and separators form a *bipartite graph* (cf. Fig. 12.1.10)

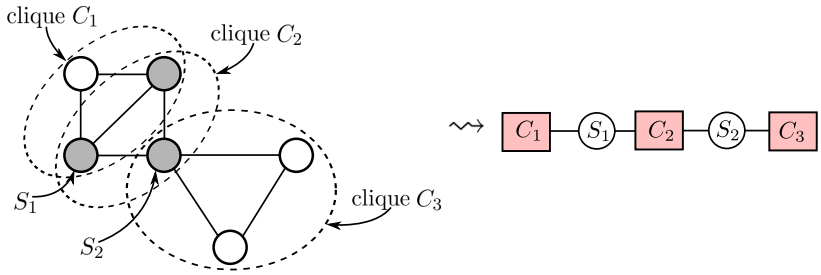


Figure 12.1.10: a decomposable graph and corresponding bipartite graph

Q 12.1.4: *What if we don't have a decomposable graph as in Fig. 12.1.7?*

- We apply triangulation. We add a *chord*, i.e. "shortcut" edges to eliminate any "circles" (i.e. cycles) of 4+ vertices.

According to Ch 9.4.3 in (Koller and Friedman, 2009), constructing the *chordal graph* is actually a heuristic of elimination ordering. It plays a role in keeping the size of cliques small. Smaller cliques allow for a more "modular" bipartite graph which leads to faster computation during inference.

12.1.3.3 Triangulation

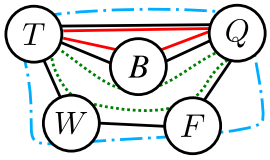


Figure 12.1.11: Identifying cycles in an undirected graph

- *Cycle*: A closed path with **no** repeated vertices other than $V_{\text{start}} = V_{\text{end}}$.
- *Chords*: Given a cycle, an edge is a chord if it connects two nodes that are non-adjacent on the cycle.
- *Chordless* cycle: **All** two **non**-adjacent vertices on the cycle are **not** joined by any edge.
- *Triangulated graph* ($\widehat{=}$ *Chordal graph*): A graph **without** chordless cycles of 4+ vertices.[3]

(too many double negatives!!!)

Triangulation

Construct a chordal graph by adding *chords* ("shortcut" edges) to all cycles of length 4+.

12.1.3.4 Triangulation: Examples

Consider the following graph with vertices A, B, C, E in Fig. 12.1.12 and add chords where necessary:

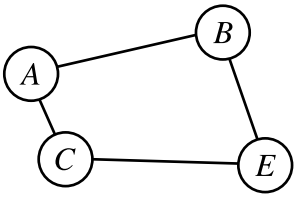


Figure 12.1.12: a non-decomposable graph

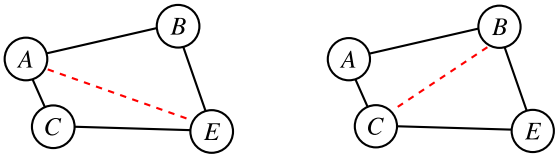
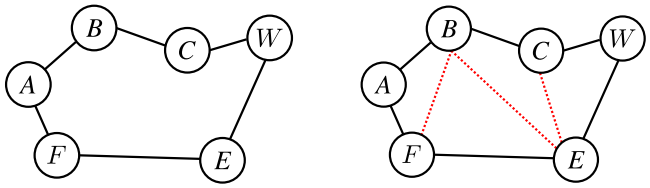


Figure 12.1.13: Possible triangulations for graph in Fig. 12.1.12

Consider the following graph with vertices A, B, C, W, E, F in Fig. 12.1.14 and add chords where necessary:

Q 12.1.5: *Is this a chordal graph (c.f. Fig. 12.1.15)? i.e. does it contain chordless cycles with 4+ vertices?*

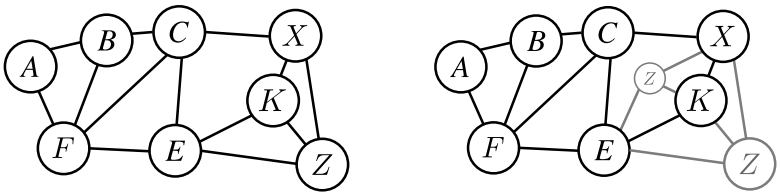
[3] cf: Definition 2.24 on Ch. 2.2.4 in (Koller and Friedman, 2009)



(a) example graph to triangulate

(b) One possible and non-unique triangulation

Figure 12.1.14: Triangulation example



(a) example graph

(b) The cycles CEKX and CEZX both require chords for the graph to be chordal.

Figure 12.1.15: Determining if a graph is chordal

12.1.4 Construction of the Junction Tree

12.1.4.1 The running intersection property

The running intersection property
All nodes on the path between two cliques, which both contain variable X, also contain X.

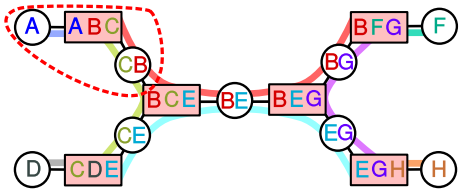


Figure 12.1.16: The running intersection property

Q 12.1.6: *Is the following a valid junction tree? (cf. Fig. 12.1.17)*

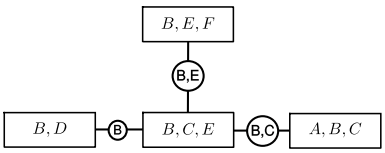


Figure 12.1.17: A valid junction tree

Q 12.1.7: *Which of the following remains a valid junction tree? (irrespective of being useful for exact inference) (cf. Fig. 12.1.18 and Fig. 12.1.19)*

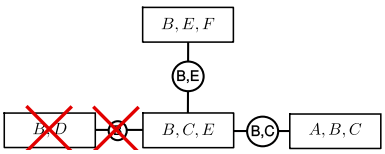


Figure 12.1.18: This graph remains a valid junction tree

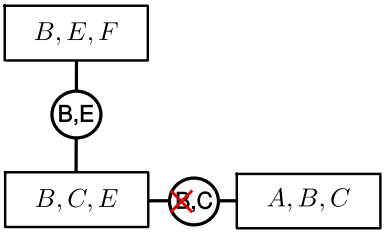


Figure 12.1.19: This graph no longer qualifies as a junction tree

12.1.5 Identify cliques and separators to form a bipartite graph

- cliques are maximally complete subgraphs (sub-cliques are automatically absorbed)
- cliques can be found by elimination

  Caution: only eliminate a node if it is not part of another clique
- requires variable ordering by (**reverse**) topological sorting: F, D, E, C, B, A

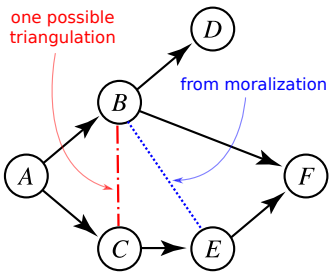


Figure 12.1.20: Example graph from the lecture.

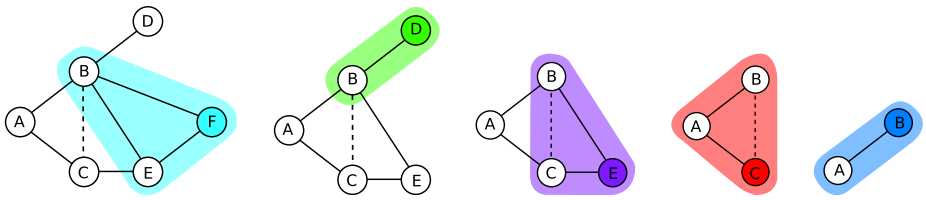


Figure 12.1.21: Identifying cliques by elimination

12.1.6 From bipartite graph to junction tree

- construct the bipartite graph:
  - edges weighted by separator size → number of nodes within separator
  - find a *maximal spanning tree*

- junction tree for inference
  - maximal spanning tree of the clique graph
  - data structure for belief propagation
  - not always unique

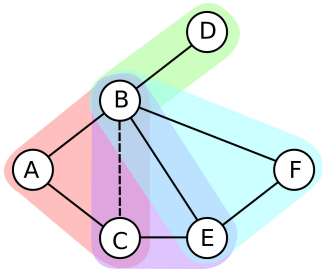


Figure 12.1.22: Identification of cliques and separators in example graph form Fig. 12.1.20

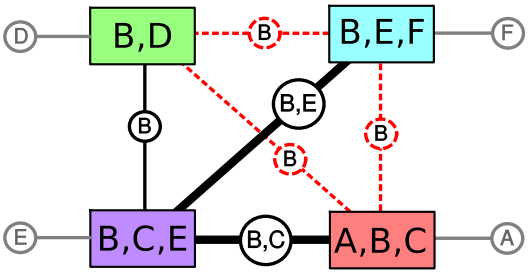


Figure 12.1.23: bipartite graph for Fig. 12.1.22

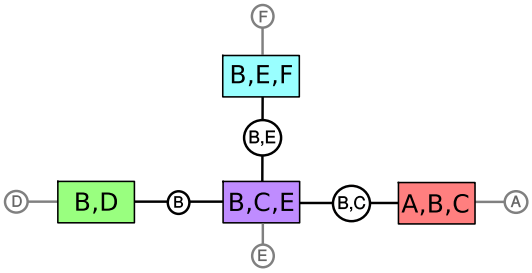


Figure 12.1.24: junction tree to bipartite graph in Fig. 12.1.23

12.1.7 Inference in a junction tree

1. clique potentials (non-negative function) for each clique (get definitions from DAG)
2. order initialization according to topological sorting (parent $\rightarrow$ child)
3. add clique potential(s) for observed evidence(s)
4. perform message passing
5. extract marginals from messages
6. perform normalization

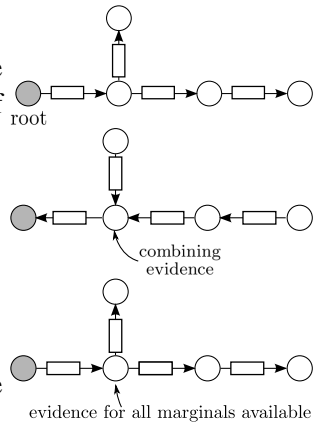
12.2 Message Passing

12.2.1 Overview

- essentially storage of local computations for faster look-up
- will be used to calculate conditional probabilities in a junction tree by only using local "simpler" computations.
- scales well with no. of variables.

12.2.2 Procedure

- first pass from root to leaves: "request"
  selecting the root (for tree structures any node can be the root, so not sensitive to choice of root node)
- second pass from leaves to root: "collect"
  collect messages from other neighbors
- a third pass can calculate all other marginals: "distribute"
  third pass from root to leaves to compute the remaining messages.



12.2.3 The sum-product algorithm

Defining the messages between the different types of nodes in a bipartite graph (e.g. separators and cliques)[4]

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m) \qquad \text{(product)}$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k) \qquad \text{(sum)}$$

- The arguments of the messages follow the variables in the separator node.
- For the *product* formula, the product runs over all clique potentials that are neighbors of the separator X_m, <u>except</u> for f_s that the message is going to.
- For the *sum* formula, the sum takes care of marginalizing over all the variables that are present in the outgoing clique potential f_s but are absent from the variables of the target separator X_i. This is weighted by the product of messages that arrive at f_s from <u>other</u> neighbors (i.e. we exclude the separator that the message is going to).

[4] for details, see Ch. 8.4.4, p. 402 in (Bishop, 2006)

12.3 Message Passing Example

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
   - in the order established by topological sorting (e.g. $A \to B \to C \to E \to D \to F$)

$f_1(A,B,C)$	$f_2(B,C,E)$	$f_3(B,D)$	$f_4(B,E,F)$
$P(C\|A)\,P(B\|A)\,P(A)$	$P(E\|C)$	$P(D\|B)$	$P(F\|B,E)$

$$P(A,B,C,D,E,F) \;=\; P(A)\,P(B|A)\,P(C|A)\,P(D|B)\,P(E|C)\,P(F|B,E) \tag{12.3.1}$$

2. modification of the clique potentials by the observed evidence
   - for each observation $Y = y^*$ find *one* f_k with $Y \in \mathcal{X}_k$
   - add a node $f_k(Y) = \begin{cases} 1, & Y = y^* \\ 0, & Y \neq y^* \end{cases}$
   - example: $\boxed{D = d^*}$ and $\boxed{E = e^*}$

3. message passing
   - begin "request" pass from arbitrary node N,e.g. $\boxed{D = d^*}$ or $\boxed{f_1}$ or ...
   - wait for all message of "collect" pass to return
   - send last "distribute" pass from N

   $\Rightarrow$ all marginals are computed simultaneously

4. calculate marginals from messages

$$P(C,\, D = d^*, E = e^*) \;=\; \sum_b \mu_{f_1 \to BC}(b,C) \cdot \mu_{f_2 \to BC}(b,C) \tag{12.3.2}$$

5. normalization yields conditional marginals

$$P(C\,|D = d^*, E = e^*) = \alpha\, \underbrace{P(C,\, D = d^*, E = e^*)}_{\tilde{P}(C)}, \qquad \alpha = \frac{1}{\sum_c \tilde{P}(c)} \tag{12.3.3}$$

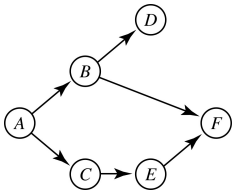


Figure 12.3.1: Original DAG

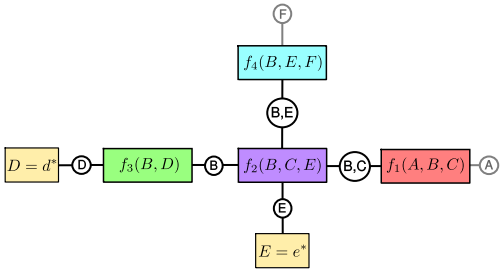


Figure 12.3.2: Adding clique potentials for observed variables to the corresponding junction tree

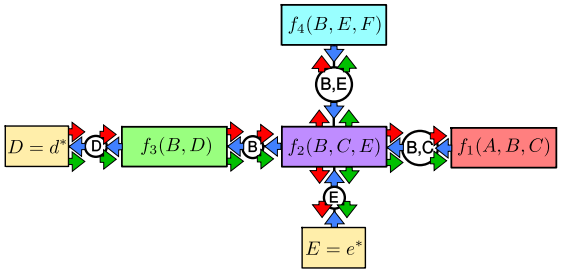


Figure 12.3.3: Flow during the 3 passes.

Define the messages by applying the sum-product-algorithm from Eq.(sum):

1. Define potentials for observed variables (in this case $D = d^*$ and $E = e^*$):
   (a) $f_5(D) = \begin{cases} 1, & D = d^* \\ 0, & D \neq d^* \end{cases}$
   (b) $f_6(E) = \begin{cases} 1, & E = e^* \\ 0, & E \neq e^* \end{cases}$
2. Select root (e.g. f_5), doesn't matter much. It will affect the order in which we the messages are defined (either in the collect or in the distribute pass).
3. collect (start at leaf node):
   (a) 'branch on the right':
      (i) $\mu_{f_1 \to B,C}(B,C) := \sum_a f_1(a,B,C) \overbrace{\cancel{\prod_{k \in \text{neigh.}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(X_k)}}^{\text{no other neighbors}}$
      (ii) $\mu_{B,C \to f_2}(B,C) := \mu_{f_1 \to B,C}(B,C) = \sum_a f_1(a,B,C)$
   (b) 'top branch':
      (i) $\mu_{f_4 \to B,E}(B,E) := \sum_f f_4(B,E,f)$
      (ii) $\mu_{B,E \to f_2}(B,E) := \mu_{f_4 \to B,E}(B,E) = \sum_f f_4(B,E,f)$
   (c) 'bottom branch':
      (i) $\mu_{f_6 \to E}(E) := f_6(E)$
      (ii) $\mu_{E \to f_2}(E) := \mu_{f_6 \to E}(E) = f_6(E)$
   (d) 'left branch':
      (i) $\mu_{f_2 \to B}(B) := \sum_{c,e} f_2(B,c,e) \cdot \mu_{E \to f_2}(e) \cdot \mu_{B,E \to f_2}(B,e) \cdot \mu_{B,C \to f_2}(B,c)$
      (ii) $\mu_{B \to f_3}(B) := \mu_{f_2 \to B}(B)$
      (iii) $\mu_{f_3 \to D}(D) := \sum_b f_3(b,D) \cdot \mu_{B \to f_3}(b)$
      (iv) $\mu_{D \to f_5}(D) := \mu_{f_3 \to D}(D)$
4. distribute (start at root node):
   (a) go right:
      (i) $\mu_{f_5 \to D}(D) := f_5(D)$
      (ii) $\mu_{D \to f_3}(D) := \mu_{f_5 \to D}(D) = f_5(D)$

(iii) $\mu_{f_3 \to B}(B) := \sum_d f_3(B, d) \cdot \mu_{D \to f_3}(d)$

(iv) $\mu_{B \to f_2}(B) := \mu_{f_3 \to B}(B)$

(b) go further to the right:

(i) $\mu_{f_2 \to B,C}(B, C) := \sum_e f_2(B, C, e) \cdot \mu_{E \to f_2}(e) \cdot \mu_{B,E \to f_2}(B, e) \cdot \mu_{B \to f_2}(B)$

(ii) $\mu_{B,C \to f_1}(B, C) := \ldots$

(c) go up: ...

(d) go down: ...

5. calculate marginals:

(a)

$$P(C,\ D = d^*, E = e^*) \quad = \quad \sum_b \mu_{f_1 \to BC}(B, C) \cdot \mu_{f_2 \to BC}(B, C) \tag{12.3.4}$$

(b) other queries you are interested in ...

6. normalize to compute conditional marginals:

$$P(C\,|D = d^*, E = e^*) = \alpha\ \underbrace{P(C,\ D = d^*, E = e^*)}_{\tilde{P}(C)}, \tag{12.3.5}$$

$$\text{where} \quad \alpha = \frac{1}{\sum_c \tilde{P}(c)} \tag{12.3.6}$$

12.4 References

References

Christopher M Bishop. *Pattern recognition and machine learning*. springer, 2006. URL http://users.isr.ist.utl.pt/~wurmd/Livros/school/Bishop%20-%20Pattern%20Recognition%20And%20Machine%20Learning%20-%20Springer%20%202006.pdf.

Daphne Koller and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.

Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Malaysia; Pearson Education Limited,, 2016.